MiniOS — Modules 1 and 2

CS2101 Operating Systems · Mini Project · **Kernel._.Crew**

Module 1 — **Anvitha** · Module 2 — **Srujana** · Modules 3–5 — Aarnavi, Advik, Krishna (upcoming)

A command-driven program for inspecting and controlling a Linux machine. Every feature answers the handbook's test: **which OS concept does this demonstrate?** Summary sheet; the full report is the accompanying document.

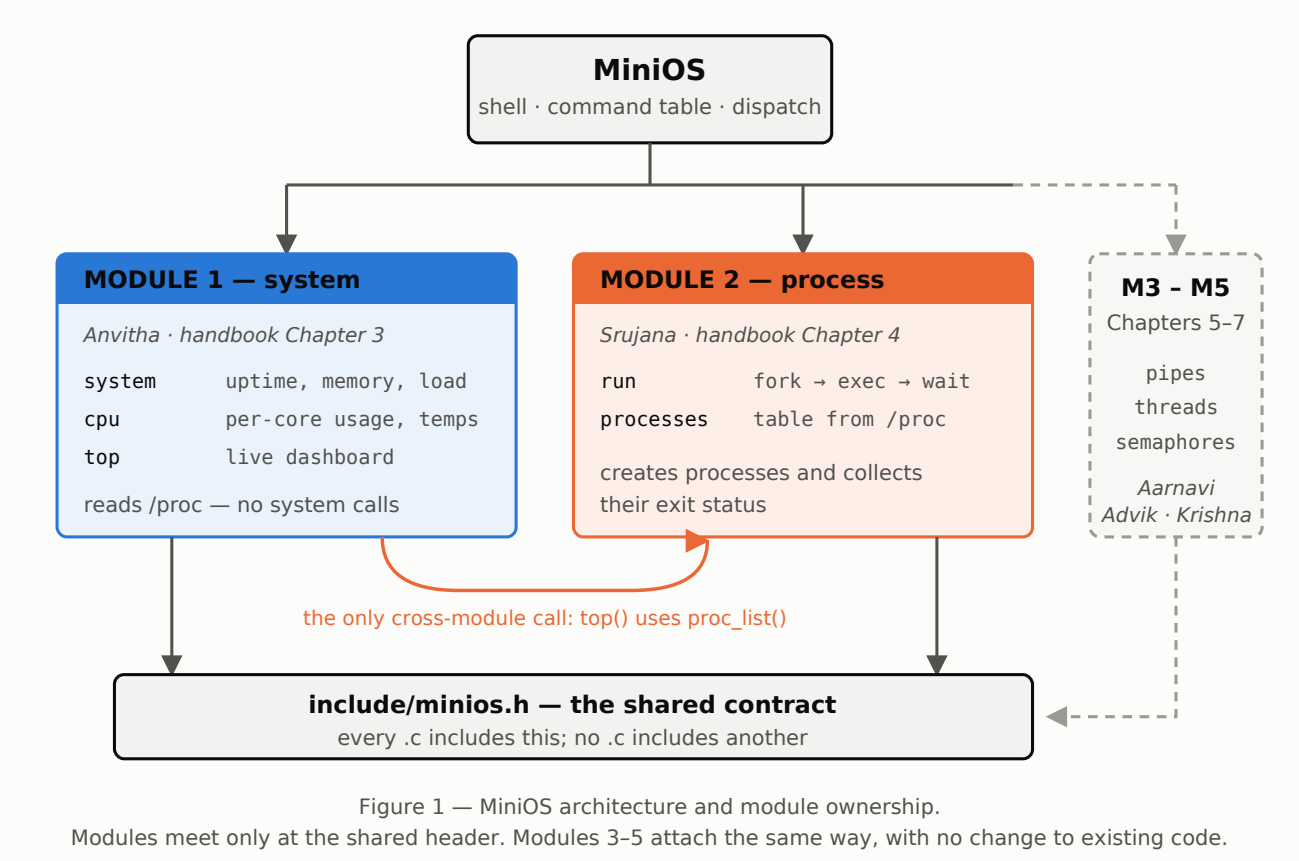


Figure 1 — MiniOS architecture and module ownership.
Modules meet only at the shared header. Modules 3–5 attach the same way, with no change to existing code.

Try it in one minute

```
make        builds MiniOS, the handbook demos and the experiments
./minios    (or ./minios demo.txt to run the whole demo with no typing)
```

Command	What it demonstrates	
`system`	`/proc` is the kernel answering, not files on disk	M1
`cpu 500`	one counter reading is meaningless — you need two, and a subtraction	M1
`top`	ANSI redraw + raw terminal + `select()` on a timer *or* a keypress	M1
`run date`	`fork` makes a process, `exec` loads a program, `wait` collects the status	M2
`processes -m`	52 fields of `/proc/<pid>/stat`, six of them parsed	M2

What the handbook asks for, and what we handed in

Seven TODOs are specified across Chapters 3 and 4. All seven are filled. Everything in the right-hand column beyond that is ours.

	Handbook	Handed in
`system`	uptime + **one** memory value	uptime, idle, 7 `meminfo` keys, swap, load averages, usage bar, host/kernel, own PID
`run`	fork → exec → waitpid	`+ fflush(NULL)` before fork, `_exit` in the child, `EINTR` retry, signal-death decoding, per-child timing

	Handbook	Handed in
`processes`	“observing processes”	+ sort by memory or CPU, row limit, children-only, state names, state summary
`help`, `quit`	both	+ `help <command>`, terminal restored on every exit path
tests	2 shell scripts	**34 checks**, three of them testing the module seam
experiments	none asked for	**6 experiments**, medians over repeated runs
—	—	`cpu` and `top` — two commands the handbook does not ask for

How the two modules were merged

Two lines of real change. Each member owns one command table, and the shell walks the tables in order, so Modules 3–5 attach without editing any existing file.

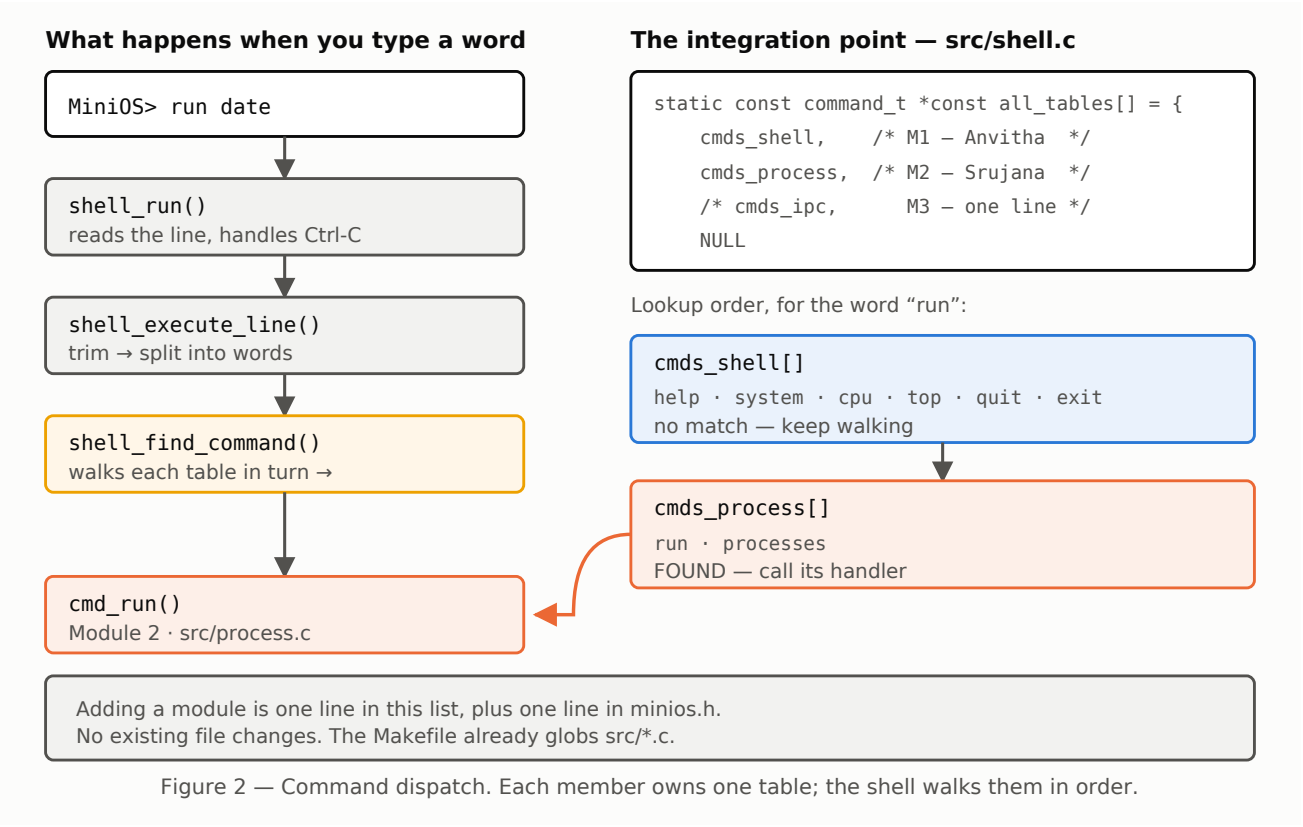


Figure 2 — Command dispatch. Each member owns one command table; the shell walks them in order.

Three conflicts had to be resolved, all documented in Part I of the full report:

- a `proc_list` defined in **both** modules — Module 2 kept it and Module 1’s copy was deleted, so there is exactly one owner;
- five command handlers listed in Module 2’s table whose source files were never handed over — commented out with a restore note, not deleted;
- one symbol defined in the wrong file, which broke any tool linking `util.c` without `shell.c`.

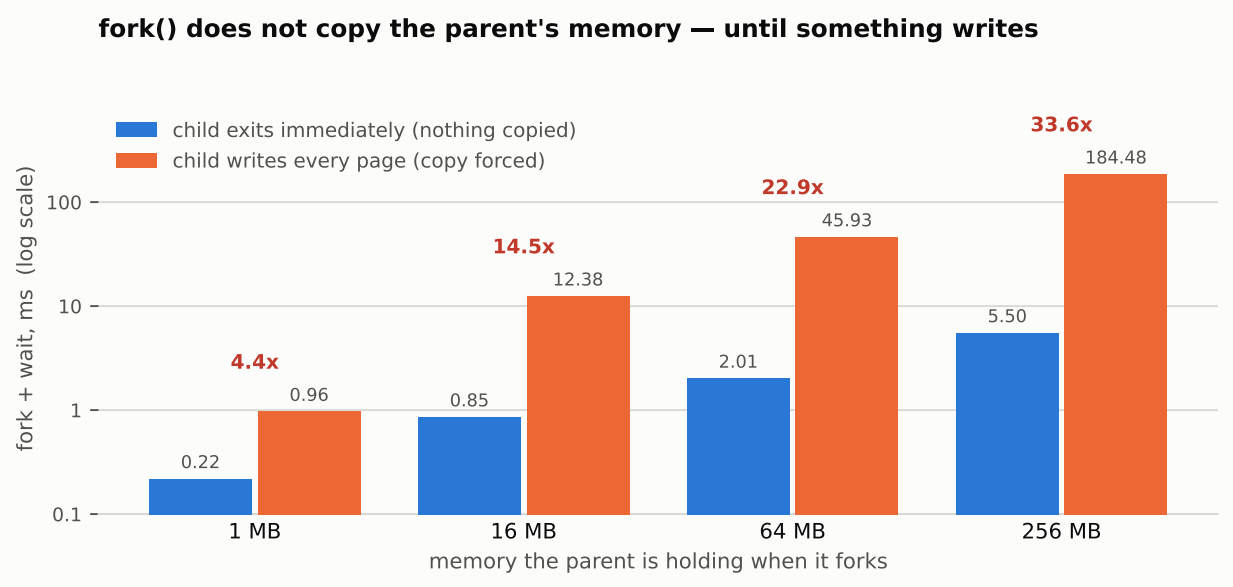
The clearest evidence the merge is real rather than cosmetic: Module 1’s `top` calls Module 2’s `proc_list()` once per frame.

Three findings, measured not quoted

Every number came from a program in `experiments/`. The raw output is in `docs/results.txt`, and the chart below is generated from that file, so the report cannot drift away from the data.

1. `exec` is the expensive half, not `fork`. Making a process costs 0.125 ms; loading a program into it costs 1.130 ms — about **9× more**, and never below 7× across seven repeats. `fork` duplicates bookkeeping; `exec` reads a file, builds a new address space and runs the dynamic linker.

2. `fork` does not copy the parent's memory. Holding 256 MB, fork costs 5.50 ms if the child exits immediately and 184.48 ms if the child writes to every page — **33.6× more**. Pages are shared read-only and copied one at a time on first write. This is exactly why `fork` followed immediately by `exec` is not the waste it looks like: `exec` discards the whole address space, and almost none of it had been copied.



3. `fork` copies the stdio buffer. A program with one `printf` before `fork` prints it **twice** when its output is piped, and once on a terminal. Buffering mode decides, not the program — and that is the bug `fflush(NULL)` prevents in `run_program()`.

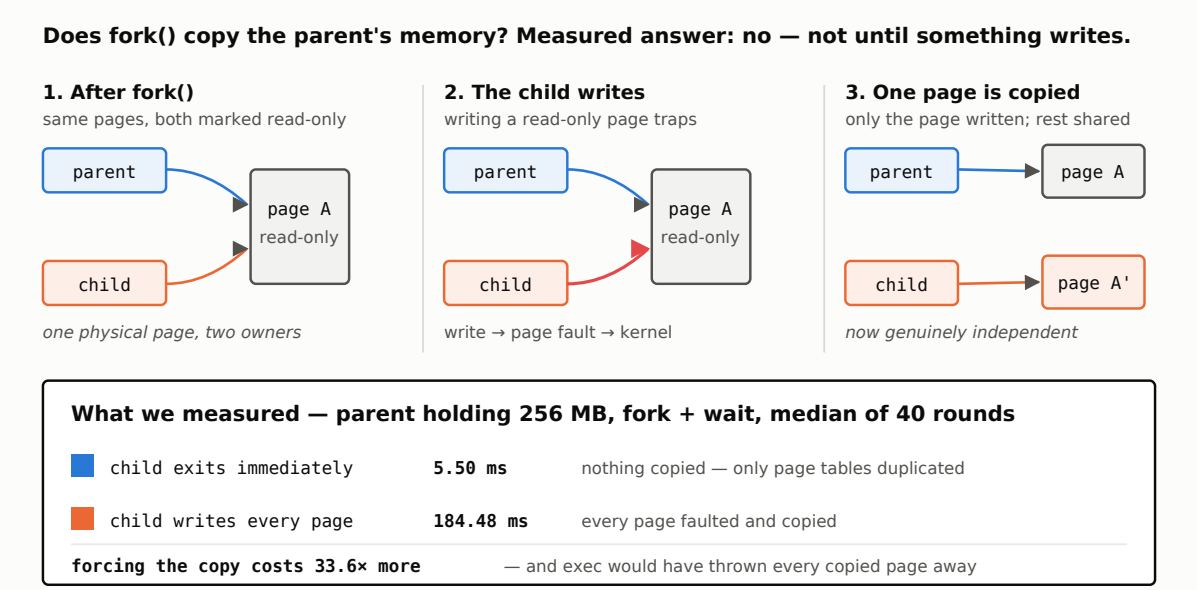


Figure 4 — Copy-on-write. This is why fork followed immediately by exec is not wasteful: exec discards the whole address space, and almost none of it had been copied.

A fourth finding, about method rather than operating systems. The first version of experiment 2 measured nothing: at `-O2`, gcc saw that nothing read the buffer afterwards and deleted the child's write loop as a dead store. The result still looked plausible — 256 MB appearing *faster* than 1 MB — which is the dangerous kind of wrong. `volatile` fixed it. We kept the episode in the report because the lesson generalises: **an optimiser can delete the thing you are trying to measure.**

Status, and what is not built

```
build          clean, zero warnings      -std=c11 -O2 -g -Wall -Wextra
```

```
tests         34 passed, 0 failed      ./tests/run_all.sh
experiments   6, all reproducible      ./experiments/run_experiments.sh
```

Not merged: Modules 3–5 — pipes (Ch 5), threads and race conditions (Ch 6–7), semaphores and the bounded queue (Ch 7) — and the optional xv6 system call. The experiments therefore stop at Chapter 4; the handbook's week 8 material needs Modules 4 and 5.

Partly handed over: Module 2's own extensions (`ps`, `wait`, `kill`, `runlimit`, `limit`) appear in its command table, but their source files were not provided. Those five rows are commented out with a four-step restore note rather than deleted.

Untested: thermal readings — this machine has no sensors. The zero-sensor path is exercised; the reading path is not.

In the full report: Part 0 team and handbook comparison · Part I integration (45%) · Part II experiments (15%) · Part III demonstration run sheet (20%) · Part IV design report (10%) · Part V viva preparation (10%). Everything regenerates from the repository: `make`, `./tests/run_all.sh`, `./experiments/run_experiments.sh`, `./docs/figures/make_chart.py`, `./docs/make_pdf.sh`.